Quarry Now Answers the Question Instead of Listing Ten Passages

quarryd turns its own search results into a short answer with numbered citations, using any OpenAI-compatible model on the same Mac or in the cloud

The menu bar app and the `quarry` CLI show the answer in one keystroke, against a local or a remote daemon

Press Release

PORTLAND, OR, April 2027. Punt Labs today added answer synthesis to Quarry, its local semantic search engine. Developers who search their own Quarry index from the macOS menu bar or the terminal can now press one key and get a three to six sentence answer, written from the passages Quarry already found, with a numbered citation on every claim. Each citation opens the source page. The answer is written by whichever OpenAI-compatible model the user points `quarryd` at: Ollama or LM Studio on the same machine for fully local use, or a hosted Haiku-class model. Chat assistants answer from documents the user uploads to them; Quarry answers from the index it already keeps, and search stays as fast as it was (see FAQ 2).

Problem

A Quarry search returns up to ten passages, and each passage can be up to 1,800 characters long. A full page of results is about 18,000 characters, roughly the length of a ten-page memo. The menu bar app shows the first 200 characters of each result. To answer a question like "which port does the staging daemon listen on, and who changed it?", the user opens results one at a time, reads each passage, and assembles the answer in their head.

Inside Claude Code this is not a problem, because the agent reads the passages for the user through Quarry's MCP tools. Outside an agent session it is: a quick lookup from the menu bar or a shell ends in reading. The usual workaround is to copy passages into a chat window, which loses the link back to the source and sends the text to whichever service the chat window uses (see FAQ 11).

Solution

The user types a question into the menu bar search panel as they do today. Results appear as they type, with no model call. When they want an answer, they press Command-Return. `quarryd` runs the same search, sends the top passages to the configured model with instructions to answer only from those passages and to cite them by number, and returns the answer together with the results it used. The answer appears in a card above the result list. Clicking `[3]` opens result 3 in the existing detail view, so every sentence is one click from its source. If the passages do not cover the question, the answer says so instead of guessing (see FAQ 7).

The same answer is available from the terminal with `quarry answer`. Because the daemon does the work, the menu bar app gets answers whether it is connected to the local `quarryd` or to a remote one, and the app never holds a model key (see FAQ 14).

Customer Quote

> *"I keep runbooks, design records and two years of incident notes in Quarry. A search used to give me ten snippets, and I'd open four of them to piece the answer together. Now I press Command-Return and get two sentences with a [2] and a [5] after them. I still click [5] before I change anything in production, and it takes me straight to the paragraph. The model runs in Ollama on my laptop, so the incident notes stay on my laptop too."*
>
> **Priya Raman**, Platform Engineer at a 40-person logistics software company

Getting Started

Users who already run Quarry need about five minutes:

1. **Pick a model.** The recommended default is OpenAI's open-weight gpt-oss-20b, run locally with `ollama pull gpt-oss:20b`. Any other OpenAI-compatible endpoint works, local or hosted.
2. **Point `quarryd` at it.** `quarry install` asks for the endpoint, model and optional key, writes them under `~/.punt-labs/quarry/`, and the daemon reads them when it starts. Synthesis stays off until this step.
3. **Ask.** Press Command-Return in the menu bar search panel, or run `quarry answer "your question"`.

Spokesperson Quote

> "Quarry already finds the right passages. The step it left to the user was reading ten of them to get one fact. That step is a small, well-bounded job for a small model: answer from these passages and say which ones. We put it in the daemon so every Quarry client gets the same answer from the same configuration, and we made the model a setting so users choose whether their notes ever leave the machine. Search stays exactly as fast as it was; the answer is one keystroke on top."
>
> **JM Freeman**, Creator of Quarry

Call to Action

Answer synthesis ships in the next minor releases of `punt-quarry` and Quarry Menu Bar, free and MIT-licensed like the rest of Quarry. The model is the user's choice and the user's cost: nothing for a local model, and about half a US cent per answer for Claude Haiku 4.5 at list price (see FAQ 5). Install or upgrade with `uv tool install punt-quarry`.

Frequently Asked Questions

External FAQs

Q1: What is answer synthesis and who is it for?

It is a short written answer to a Quarry search, with numbered citations back to the results. It is for people who already keep their documents, notes and captured research in Quarry and look things up from the menu bar app or the terminal, outside an agent session. The job it does is the last step of a lookup: turning ten relevant passages into the one or two facts the user wanted.

Q2: How is this different from pasting the results into ChatGPT or Claude?

The result is similar; the work is gone. There is no copying, the citations stay linked to the source pages, and the user decides once, in configuration, which model sees their passages. Hosted notebook products such as NotebookLM also answer with citations [1], but they answer from documents uploaded to that service. Quarry answers from the local index that already holds the user's files, web captures and session notes.

Q3: I use Quarry through Claude Code. Does this change anything for me?

No. In Claude Code the agent already reads Quarry's results and writes the answer itself. Quarry's MCP tools keep returning passages, because a pre-written answer would cost the agent tokens and hide the source text it reasons over (see Feature 11).

Q4: How do I get started?

Run `ollama pull gpt-oss:20b`, or choose another OpenAI-compatible endpoint. Run `quarry install`, which records the endpoint, model and optional key under `~/.punt-labs/quarry/`, and restart `quarryd`. Then press Command-Return in the menu bar search panel or run `quarry answer`. Until a model is configured, the menu bar app hides the Answer action and the CLI command reports that synthesis is not configured.

Q5: How much does it cost?

Quarry is free. The model costs whatever the chosen endpoint charges. A local model costs nothing beyond electricity. For a hosted model, one answer sends about eight passages plus instructions, roughly 4,000 input tokens at four characters per token, and returns about 250 output tokens. At Claude Haiku 4.5 list prices of $1 per million input tokens and $5 per million output tokens [2], that is about $0.005 per answer, or $5 for a thousand answers.

Q6: What happens to my data?

With a local model, nothing leaves the machine. With a hosted model, each answer sends that question and the top passages for it to the configured endpoint, and nothing else: the index, other documents and the search history stay local. Every answer response names the endpoint host that wrote it, and the menu bar card shows that host under the answer. Quarry stores no answers and sends nothing to Punt Labs. Session captures are passed through Quarry's secret scrubber when they are ingested, but the rest of the user's corpus is sent as indexed, so users with sensitive files should choose a local model.

Q7: What if the answer is wrong?

The model is told to answer only from the numbered passages, to cite each claim, and to say when the passages do not answer the question. Quarry then checks the citations: a number that does not match a returned result is removed before the answer reaches the user. The results are always shown under the answer, so the user can check any sentence in one click. A small model can still misread a passage; the citations make that visible rather than preventing it (see FAQ 17).

Q8: I connect the menu bar app to Quarry on another machine. Whose model answers?

The remote daemon's. The model call runs on the machine that runs `quarryd`, with that machine's configuration and key. The menu bar app sends the question with the same bearer token it already uses for search and stores no model settings of its own. If the remote daemon has no model configured, the Answer action is hidden.

Q9: Will this make search slower?

No. Search-as-you-type keeps calling the existing search endpoint with no model in the path. An answer is a separate request that runs only when the user asks for one, and if the model is slow or down, the answer card shows the error while search results keep working.

Internal FAQs

Value & Market

Q10: What is the total addressable market?

Quarry is free and open source, so the relevant market is Quarry's own users, and the feature is judged by whether it makes Quarry more useful to them. `punt-quarry` had 469 PyPI downloads in the month to 2026-10-25 [3]; downloads include CI and repeat installs, so active users are fewer. The menu bar app is macOS-only and has no download or usage counter, so its share of that population is unknown. The upper bound on reach is every Quarry user who searches outside an agent session, through the menu bar app or the CLI.

Q11: What evidence do we have that users want this?

Little beyond the author's own use. The request comes from the Quarry author using the menu bar app daily, which is the weakest kind of evidence: the customer is the team. Two indirect signals support it. First, the size of the results: ten passages of up to 1,800 characters, of which the menu bar shows 200, leave reading as the main cost of a lookup. Second, products built on the same pattern, such as NotebookLM [1], answer from a user's sources with citations. Neither shows that Quarry users want an answer more than they want the passages.

Q12: Who are the alternatives and why would a user pick this?

The real alternatives are habits. (1) Reading the results, which costs nothing and needs no model. (2) Asking an agent that has Quarry's MCP tools, which already synthesizes but means opening a session for a quick lookup. (3) Pasting passages into a chat assistant, which loses citations and sends text to that service. (4) Hosted notebook products [1], which require uploading documents and do not see Quarry's captures. Quarry's advantage is position: it already holds the corpus and the ranked passages, so the answer is one model call away. Any of these alternatives could add something similar; none of them has the user's local index.

Q13: What is the next step to validate the vision?

Ship the daemon route, the CLI command and the menu bar card together, then run the eval in FAQ 16 against the recommended default and record the metrics in FAQ 22 during normal use. The first numbers to check are how often an answer says the passages do not cover the question, and how often the user still opens a result.

Technical

Q14: Why put synthesis in `quarryd` instead of the menu bar app?

Three reasons. Every client gets the same behavior: the menu bar app, the CLI and any future client call one route, and a remote menu bar connection gets answers without its own model setup. Keys live in one place, on the machine that runs the daemon; the menu bar app has no settings screen today and would otherwise need one plus Keychain storage. And the daemon already has the passages in memory when it searches, so it can send them to the model without a second round trip. The cost is that the daemon now makes outbound HTTP calls and holds a model key, and a remote daemon spends its owner's tokens on behalf of every caller holding its bearer token.

Q15: Why not run a generative model inside `quarryd`, like the embedding model?

The embedding model is a small encoder that ONNX Runtime runs on the CPU inside the daemon. A generative model of useful quality is many times larger, would add gigabytes of download and resident memory to a daemon that is always running, and on macOS would run on the CPU because Quarry's ONNX path has no GPU provider there. Generation also needs chat templates, a sampling loop and stop handling that Quarry would have to own. Ollama, LM Studio and llama.cpp already serve local models behind an OpenAI-compatible API [4, 5, 6] and use the Mac's GPU, so one HTTP client in Quarry gets local synthesis without Quarry managing a model.

Q16: Which model is the default, and why?

OpenAI's gpt-oss-20b, served by Ollama. The criteria are a US open-weight model, small and fast enough for a laptop, with some reasoning and no need for tool use. gpt-oss-20b has 21 billion parameters of which 3.6 billion are active per token, an Apache 2.0 license, and a configurable reasoning effort of low, medium or high [7]. Quarry sets it to low. It is a 14 GB download in Ollama and runs within 16 GB of memory [8, 7], which is the floor: on machines with less, the recommendation is IBM's Granite 3.3, also Apache 2.0 with an optional thinking mode, at 4.9 GB for 8B parameters or 1.5 GB for 2B [9, 8]. The eval in Feature 8 confirms the choice before release; answer latency on an M-series Mac has not been measured for either.

Q17: What are the major technical risks?

Answer quality from small local models (high, unknown). A 1–4B parameter model may cite the wrong passage or merge two passages into a claim neither makes. Mitigation: run Quarry's eval fixtures through gpt-oss-20b, Granite 3.3 and one hosted model, scored for citation correctness by hand, before release. **Latency on a laptop (medium, unknown).** A local model's time to a full answer on an M-series Mac is not measured; the spike records it. **Provider compatibility (medium).** "OpenAI-compatible" varies between servers. Anthropic describes its compatibility layer as intended for testing and not a long-term or production-ready solution

for most use cases [10], so the client must stay within the core chat completions fields and treat provider differences as test cases. **Key delivery to a background daemon (low).** `quarryd` runs as a service, so keys exported in a shell do not reach it. The key follows the daemon's existing pattern: written under `~/.punt-labs/quarry/` at install time and read when the daemon starts. **Cost exposure on a shared daemon (low for personal use).** Any holder of a remote daemon's bearer token can spend its model budget; a per-daemon request limit bounds this.

Q18: What dependencies exist on other systems?

The OpenAI chat completions request and response format, as implemented by the endpoint the user chooses. `httpx`, already a Quarry dependency, for the outbound call. The existing `SearchService`, authorization and collection filtering inside `quarryd`, reused unchanged. On the client side, the menu bar app's existing `QuarryClient` and detail view. No new third-party service is required: a user with no model configured sees no change.

Q19: What is the delivery order?

(1) Daemon: configuration written by `quarry install`, the OpenAI-compatible client, the answer route with citation checking, the OpenAPI update and tests against a fake endpoint. (2) Clients, in parallel with the daemon once the response format is fixed: `quarry answer`, and the menu bar Answer action, answer card with clickable citations, and endpoint host label. (3) The model eval in FAQ 16. (4) Streaming, if the measured latency calls for it. If scope compresses, streaming is cut first, then the CLI command; the daemon route and the menu bar card are the feature.

Q20: What breaks first if it is used heavily?

The model endpoint. Each answer is one search plus one model call, so the daemon's own load barely changes, but a local model serves one request at a time and a hosted one has rate limits. The daemon needs a timeout on the model call, a cap on concurrent answer requests, and an error the client can show without blocking search.

Business

Q21: What is the revenue model?

None. Quarry is free and MIT-licensed. The value to Punt Labs is a menu bar app that is useful on its own for quick lookups, which brings users to the rest of Quarry. Punt Labs pays no model costs: the user's endpoint and key are used for every call.

Q22: How will we measure success?

Quarry's local recall telemetry already records searches; answer requests would be recorded the same way, on the user's machine only. (1) Answer requests as a share of searches, per active user per week; below 10% after the first month of use suggests users prefer passages. (2) Share of answers that say the passages do not cover the question; above 30% points at retrieval, not synthesis. (3) Citation clicks per answer, a sign that users check answers. (4) Median time to a full answer for local and hosted models; above five seconds makes streaming a must. (5) Model call error rate.

Q23: Why now?

Three things are in place. The common local model servers expose the same OpenAI-compatible API [4, 5, 6], so one client covers local and hosted models. Haiku-class models answer a 4,000-token question for about half a cent [2]. And Quarry is now daemon-first, with every client going through `quarryd`, so a feature added there reaches the menu bar app, the CLI and remote connections at once.

Q24: What are we not building?

Multi-turn chat, a synthesis tool in MCP, a model bundled inside Quarry, model settings in the menu bar app, and answers that run on every keystroke. The Feature Appendix lists each with its reason.

Risk Assessment

Risk	Rating	Assessment
Value	Medium	Demand evidence is the author's own use (FAQ 11). Usage metrics after release are the test (FAQ 22).
Usability	Medium	One keystroke to use, but setup asks the user to choose and run a model and edit configuration. Users without a model see no change.
Feasibility	Medium	The plumbing is one route and one HTTP call over existing search. Answer quality and latency of the default model on a laptop are unmeasured (FAQ 16).
Viability	Low	No cost to Punt Labs; users pay their own model costs. Main exposure is a shared remote daemon spending its owner's tokens.

Feature Appendix

Must Do

- **F1.** **Answer route in `quarryd`**: a `/v1` route separate from search that reuses `SearchService`, authorization and collection filtering, and returns the answer with the results it cites
- **F2.** **OpenAI-compatible client**: one chat completions call over `httpx` with a timeout and a concurrency cap; works with Ollama, LM Studio, llama.cpp and hosted endpoints
- **F3.** **Off until configured**: endpoint URL, model and key are written under `~/.punt-labs/quarry/` by `quarry install` and read at daemon start; with none set, the route reports "not configured" and the daemon's status tells clients to hide the feature
- **F4.** **Grounded prompt and citation check**: numbered passages, answer-only-from-sources instructions, an explicit "not covered" reply, and server-side removal of citation numbers that match no result
- **F5.** **Endpoint disclosure**: every answer names the endpoint host that wrote it, and clients display it
- **F6.** **Menu bar Answer card**: an explicit Command-Return or button trigger, a card above the results, clickable citations that open the existing detail view, and cancellation when the query changes

Should Do

- **F7.** **`quarry answer`**: the CLI command, cheap once the route exists
- **F8.** **Answer quality eval**: run Quarry's eval fixtures through gpt-oss-20b, Granite 3.3 and a hosted model and score citation correctness, to confirm the default
- **F9.** **Answer telemetry**: record answer requests, "not covered" replies and latency in Quarry's local recall telemetry
- **F10.** **Streaming**: show the answer as it is written, if measured latency makes waiting noticeable

Won't Do

- **F11.** **Synthesis in MCP**: the MCP caller is already a model; a pre-written answer costs it tokens and hides the passages it reasons over
- **F12.** **A bundled generative model**: gigabytes of download and memory in an always-on daemon, CPU-only on macOS, and a sampling loop to own, when local servers already exist (FAQ 15)
- **F13.** **Model settings in the menu bar app**: keys and endpoints belong to the daemon that makes the call; the app stays a thin client
- **F14.** **Answers on every keystroke**: search-as-you-type would fire a model call for every half-typed query
- **F15.** **Multi-turn chat**: follow-up questions turn a lookup tool into a chat client, a different product with its own state and UI
- **F16.** **Native, non-OpenAI provider APIs**: one request format keeps the client small; providers without an OpenAI-compatible endpoint can be reached through a gateway

References

[1] Google. *NotebookLM*. URL: https://notebooklm.google/ (visited on 10/25/2026).

[2] Anthropic. *Pricing: Claude API*. Claude Haiku 4.5: $1 per million input tokens, $5 per million output tokens. URL: https://docs.claude.com/en/docs/about-claude/pricing (visited on 10/25/2026).

[3] PyPI Stats. *punt-quarry recent downloads*. 469 downloads in the last month, 166 in the last week, as of 2026-10-25. URL: https://pypistats.org/packages/punt-quarry (visited on 10/25/2026).

[4] Ollama. *OpenAI compatibility*. Chat completions at /v1/chat/completions on port 11434. URL: https://docs.ollama.com/api/openai-compatibility (visited on 10/25/2026).

[5] LM Studio. *OpenAI Compatibility Endpoints*. Chat completions at /v1/chat/completions on port 1234. URL: https://lmstudio.ai/docs/app/api/endpoints/openai (visited on 10/25/2026).

[6] ggml-org. *llama.cpp HTTP Server*. OpenAI-compatible chat completions API. URL: https://github.com/ggml-org/llama.cpp/blob/master/tools/server/README.md (visited on 10/25/2026).

[7] OpenAI. *gpt-oss-20b model card*. 21B parameters with 3.6B active; Apache 2.0; configurable reasoning effort; runs within 16GB of memory. URL: https://huggingface.co/openai/gpt-oss-20b (visited on 10/25/2026).

[8] Ollama. *Model library: gpt-oss and granite3.3*. gpt-oss:20b 14GB; granite3.3:8b 4.9GB; granite3.3:2b 1.5GB. URL: https://ollama.com/library/gpt-oss (visited on 10/25/2026).

[9] IBM. *Granite-3.3-8B-Instruct model card*. 8B parameters; Apache 2.0; optional thinking mode. URL: https://huggingface.co/ibm-granite/granite-3.3-8b-instruct (visited on 10/25/2026).

[10] Anthropic. *OpenAI SDK compatibility*. Described as primarily for testing and not a long-term or production-ready solution for most use cases. URL: https://docs.claude.com/en/api/openai-sdk (visited on 10/25/2026).